



Lesson 2

Procedures, Functions and Structure charts



What is in this lesson?

Procedures

Functions

Documenting code

Local and Global scoping

Methods, attributes and classes

Hierarchy charts

Procedures

Procedures

- ▶ **Procedure** is a sequence of instructions designed to perform a specific task.
- ▶ Normally, it is in the form of a block of code.
- ▶ Any interaction a **procedure** performs with the outside world is called a **side effect**.

Passing Arguments

- ▶ **Procedures** can take **arguments**.
- ▶ **Arguments** are data that is sent (**passed**) from one block of code to another.
- ▶ **Arguments** are given names (like a variable name) in the receiving procedure.
- ▶ A procedure can take multiple **arguments**.
- ▶ Variables can be used to pass values to a procedure.

Functions

Divide and Conquer [1, Ch. 5]

- ▶ Most programs are large enough to be broken down into smaller pieces known as **functions**.
- ▶ A program that has been written with each task in its own function is called a modularised program.

- ▶ Simpler code
- ▶ Code reuse
- ▶ Better testing
- ▶ Faster development
- ▶ Teamwork

This program is one long, complex sequence of statements.

statement
statement
statement
statement
statement
statement
statement
statement
statement
statement
statement
statement
statement
statement
statement
statement
statement
statement
statement
statement
statement

In this program the task has been divided into smaller tasks, each of which is performed by a separate function.

```
def function1():  
    statement  
    statement  
    statement
```

function

```
def function2():  
    statement  
    statement  
    statement
```

function

```
def function3():  
    statement  
    statement  
    statement
```

function

```
def function4():  
    statement  
    statement  
    statement
```

function

Figure: Using functions to **divide and conquer** a large task

Functions

Function

A **function** is a group of statements that exist within a program for the purpose of performing a specific task.

```
# This program demonstrates a function.  
# First, we define a function named message.  
def message():  
    print('I an Arthur,')  
    print('King of the Britons.')  
  
# Call the message function.  
message()
```

These statements cause the message function to be created.

This statement calls the message function, causing it to execute.

The diagram illustrates the relationship between the function definition and its call. A bracket on the left groups the function definition lines (from the first comment to the end of the function body), with an arrow pointing to the text "These statements cause the message function to be created." Another arrow points from the text "This statement calls the message function, causing it to execute." to the function call line, `message()`.

Figure: The function definition and the function call

Function with arguments [1, Ch. 5]

Argument

An **argument** is any piece of data that is passed into a function when the function is called.

Parameter

A **parameter** is a variable that receives an argument passed into a function.

```
1 def main():
2     value = 5                                # value is a variable
3     double_value = get_double(value)         # value is passed as an argument
4
5     def get_double(number):                  # number is a parameter
6         result = number * 2
7         print(result)                        # This will create a side effect
8
9         return result
10
11 main()
```

Void Functions and Value-Returning Functions

[1, Ch. 5]

- ▶ A **Void function** executes the statements it contains and then terminates **without returning a value**.
- ▶ A **Value-returning function** executes the statements it contains and then **returns a value** back to where it was called.
 - ▶ A value-returning function has a **return** statement that returns a value back to the part of the program that called it.

```
1  # main is a void function
2  def main():
3      value = 5                    # value is a variable
4      double_value = get_double(value)  # value is passed as an argument
5
6  # get_double is a value-returning function
7  def get_double(number):          # number is a parameter
8      result = number * 2
9      print(result)               # This will create a side effect
10
11     return result
12
13  main()
```

Indentation in Python [1, Ch. 5]

- ▶ In Python, each line in a block must be indented.
- ▶ In a block, each line has to be indented with the same number of spaces. Otherwise, an error will occur.

Documenting code

Documenting code

- ▶ Comments explain the code in plain words.
- ▶ Comments are ignored during compilation and execution.

Use comments before functions and procedures to explain:

- ▶ How to use it
- ▶ What arguments it takes
- ▶ What it returns

Try to avoid explaining how your code works.

Local and Global scoping

Local Variables [1, Ch. 5]

Concept

A **local variable** is created inside a function and cannot be accessed by statements that are outside the function.

- ▶ An error will occur if a statement in one function tries to access a local variable of another function.
- ▶ Different functions can have local variables with the same names.

Scope and Local Variables [1, Ch. 5]

A variable's **scope** is the part of a program in which the variable may be accessed.

```
1  # This program demonstrates two functions that
2  # have local variables with the same name.
3  def main():
4      # Call the texas function.
5      texas()
6      # Call the california function.
7      california()
8
9  # Definition of the texas function. It creates
10 # a local variable named birds.
11 def texas():
12     birds = 5000
13     print(f'texas has {birds} birds.')
14
15 # Definition of the california function. It also
16 # creates a local variable named birds.
17 def california():
18     birds = 8000
19     print(f'california has {birds} birds.')
20
21 # Call the main function.
22 main()
```

Global Variables [1, Ch. 5]

Concept

A **global variable** is accessible to all the functions in a program.

```
1  # Create a global variable.
2  my_value = 10
3
4  # The show_value function prints
5  # the value of the global variable.
6  def show_value():
7      print(my_value)
8
9  # Call the show_value function.
10 show_value()
```

Restrict the use of global variables [1, Ch. 5]

- ▶ Any statement in the program can change the value of a global variable.
- ▶ Functions that use global variables are usually dependent on those variables.
- ▶ Global variables make a program hard to understand.

Global Constants [1, Ch. 5]

Concept

A **global constant** is a global name that references a value that cannot be changed.

```
1  # The following is used as a global constant to represent the Pi.
2  PI = 3.14159265359
3
4  def main():
5      rad = float(input("Enter the radius of the circle: "))
6      circle_area(rad)
7      circle_perimeter(rad)
8
9  # The circle_area function accepts the radius as an argument, then displays the
10 # area of the circle with the given radius.
11 def circle_area(radius):
12     area = PI * radius**2
13     print("The area of the circle is: ", area)
14
15 # The circle_perimeter function accepts the radius as an argument, then displays
16 # the perimeter of the circle with the given radius.
17 def circle_perimeter(radius):
18     perimeter = 2 * PI * radius
19     print("The perimeter of the circle is:", perimeter)
20
21 main()
```

Methods, attributes and classes

Procedural programming and Object-oriented programming [1, Ch. 10]

Procedural programming

Procedural programming is a method of writing software, centered on the **procedures** or actions that take place in a program.

Object-oriented programming

Object-oriented programming (OOP) is centered on **objects**.

Methods, Attributes and Classes

- ▶ **Methods**: an OOP term for both functions and procedures.
- ▶ **Attributes**: variables, also functions or methods that might be used to give values as though you were accessing a variable.
- ▶ **Class**: in this unit, **class** is utilised as a way of grouping variables and methods.
- ▶ **Objects**: are created from abstract data types that encapsulate data and functions.

Hierarchy charts

Hierarchy Charts

One way to think about and represent modularised programs is using a **hierarchy chart/ structured chart**.

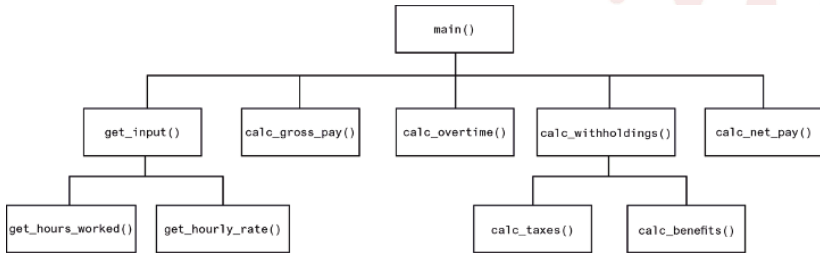


Figure: A hierarchy chart

- ▶ **Hierarchy chart** gives a visual representation of the relationships between functions.
- ▶ It uses boxes to represent each function in a program.
- ▶ The boxes are connected in a way that illustrates the functions called by each function.
- ▶ A hierarchy chart does not show in detail what is performed inside a function.

Recap

In this lecture, we've learned:

- ▶ Procedures and Functions, and how to pass arguments to a procedure/ function
- ▶ Void functions and Value-returning functions
- ▶ Documenting code
- ▶ Local and Global scoping
- ▶ Methods, Attributes and Classes
- ▶ Structure charts

Further Reading



Gaddis, Tony *Starting out with Python*. Pearson Education Limited. Sixth Edition, 2024.

Any Question?